CSE515: Advanced Communication Networks and System

Project Report

on

Internet of Things in Healthcare

Aman Bhaskar – 20124006

Harsh Lalit Parmar – 20124022

Womesh Ratre – 20124050

Swastik Singh - 20124048

Introduction

The Internet of Things (IoT) is an exciting field that is transforming society by combining advances in wireless networking with tiny sensors, devices, actuators, and embedded microprocessors. This integration meets the needs of various applications around the world. The idea behind wireless sensor networks can be summed up with a simple formula:

Sensing + Processing + Transfer and Receiving = Many Applications.

One example of IoT in action is a remote healthcare monitoring system, which is a smaller version of traditional hospital monitoring systems. These systems can track a patient's vital signs from a distance with similar efficiency. Previously, disease detection systems were large, complicated, and consumed a lot of power, requiring skilled operators.

However, advancements in semiconductor technology have produced smaller, faster, more energy-efficient, and affordable sensors and microcontrollers.

Motivation

The motivation behind this project stems from the growing need for efficient and accessible healthcare solutions, especially in the context of remote patient monitoring. With an aging population and the rising prevalence of chronic diseases, healthcare systems are under increasing pressure to provide timely and effective care. Traditional monitoring methods often fall short, as they require frequent hospital visits and rely heavily on in-person interactions. This can lead to delays in treatment and heightened anxiety for patients and their families. By leveraging IoT technology to create a live sensor feed accessible via a web interface, this project aims to empower patients and their loved ones with real-time health information, fostering a sense of control and involvement in their care.

Internet of Things for healthcare

The Internet of Things (IoT) is a crucial field recognized as a key driver for automation and control across nearly every industry. It has been extensively researched in recent years. IoT-enabled health monitoring systems in rural areas have a significant impact on traditional healthcare models. As the costs of medical care, especially for chronic conditions, continue to rise, many people are affected. The growing elderly population further strains medical facilities and services.

IoT offers various benefits in healthcare through wearable devices and medical applications. Many hospitals use IoT technology to track the location of medical teams, staff, and patients. An IoT based e-Healthcare system allows healthcare providers to carry patient information with them via mobile apps, enabling better treatment and monitoring. Any remote healthcare system must ensure continuous data analysis to effectively support patients, providing comprehensive diagnostic information through sensors.

Related Works

- **Gupta D. (2020)** outlines a healthcare-monitoring system using the IoT for obese patients . The prototype is a fully functional device that measures body characteristics such as HR, SpO2, BP, and BT. [1]

- To help physicians diagnose and monitor their patients' health status, **Alamsyah and**

Ikhlayel (2020) developed a monitoring system based on an IoT that can detect vital signs . The system uses sensors to collect vital signs such as HR, BP, and BT. [2]

- **A. D. Acharya and S. N. Patil (2020)** designed and implemented an IoT-based smart medical kit for critical medical conditions . [3]

- **Kishor and Chakraborty(2022)** designed a healthcare model using seven classifying algorithms . Nine different disease related datasets were organized based on classifications. [4]

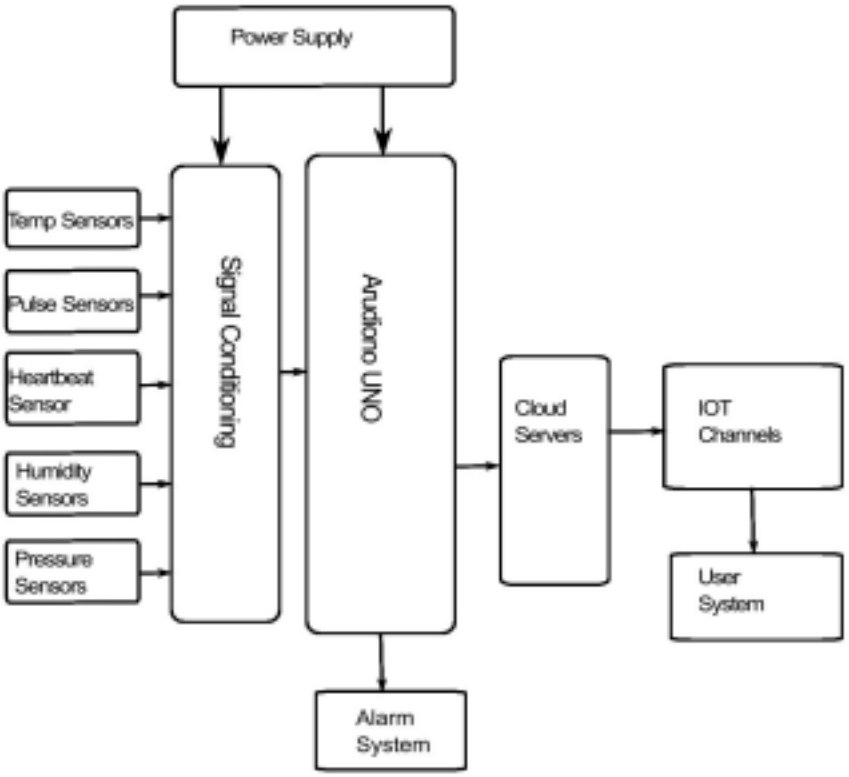
Problem statement

Current remote monitoring systems lack a user-friendly interface that allows patients and their families to access real-time sensor data, resulting in a disconnect between the patient's health status and their support network. Moreover, healthcare providers may not have immediate access to this data, hindering timely decision-making and personalized care.

Solution

This project proposes the development of an integrated IoT-based pipeline that enables patients and their families to view live sensor feeds through a web interface. We also try to integrate AI models in our system to aid the patient in interpreting the data from the live feed. By bridging the communication gap, this solution aims to improve patient engagement, enhance care coordination, and optimize health outcomes in remote patient monitoring scenarios. The main idea of the proposed system is to design a Patient Monitoring System which will able to establish two way communication i.e. not only the patient's data will be sent to the doctor through SMS and email on emergencies, but also the doctor can send required suggestions to the patient or guardians through SMS or Call or Emails

1. Block Diagram



The block diagram of system (fig. 1) contain body temperature and heart rate, ECG, blood pressure, humidity sensor etc. This health monitoring sensors are used to collect health related data i.e. for data acquisition. Communication process completed with the help of controller, which send the data on the internet. Data processing will beoccursat server. All data collected and aggregated at server point. To display patient's health related information in understandable format it can be shown on web page i.e. data management, which can be used by, related patient's doctor, parents or any other person who is taking care of patient.

2. System Design

2.1 Architecture Overview

The IoT-based patient health monitoring system comprises several key components that work together to collect, transmit, and display patient health data. The overall architecture is divided into three main layers:

1. Sensor Layer: This layer consists of various sensors that collect vital health parameters from patients. These sensors measure pulse rate, blood pressure, blood glucose levels, and body temperature.

2. Processing Layer: An Arduino microcontroller acts as the central processing unit. It

gathers data from the sensors, processes it, and prepares it for transmission to the cloud. The Arduino is responsible for managing the data flow and ensuring reliable communication with the cloud service.

3. Cloud Layer: Data is sent to a cloud platform where it is stored and processed. This layer facilitates data access for both patients and healthcare providers through a web interface. The cloud service also allows for data analytics, storage, and management of user accounts.

2.2 Hardware Components

2.2.1 Sensors:

- Pulse Sensor: Monitors heart rate using photoplethysmography.

- Blood Pressure Sensor: Measures systolic and diastolic blood pressure. **-**

Blood Glucose Sensor: Utilizes electrochemical technology to determine glucose levels. **- Temperature Sensor**: Measures body temperature using a thermistor or thermocouple.

2.2.2 Arduino Microcontroller:

- Model: Arduino Uno or Arduino Nano.

- Function: Collects data from sensors and processes it for cloud transmission.

2.2.3 Communication Module:

- Wi-Fi Module: ESP8266 or similar, used for connecting the Arduino to the internet. **-**

Power Supply: Battery pack or USB power supply to ensure continuous operation of the

system.

2.3 Software Components

2.3.1 Programming Language:

- C++ for Arduino programming, enabling direct interaction with sensors and data processing.

2.3.2 Libraries:

- Sensor Libraries: Specific libraries for interfacing with each sensor (e.g., `Adafruit_Sensor`, `DHT`, etc.).
- Wi-Fi Library: For connecting the Arduino to the internet (e.g., `ESP8266WiFi`).

2.3.3 Cloud Platform:

- Option: AWS IoT, Firebase, or another cloud service for data storage and management. - Data Protocol: MQTT or HTTP for sending data from the Arduino to the cloud.

2.3.4 Web Development Framework:

- HTML/CSS and JavaScript for the front-end development of the web interface. - Frameworks like React or Angular can also be considered for a more dynamic user experience.

2.4 User Interface Design

The web interface is designed for ease of use for both patients and healthcare providers:

2.4.1 Patient Dashboard :

- Real-time display of health metrics.
- Historical data charts and trends.
- Alerts for abnormal readings.

2.4.2 Doctor Dashboard:

- Access to multiple patient data.
- Analysis tools to compare readings over time.
- Communication tools for direct contact with patients.

2.5 Security Measures

To protect sensitive health data, the following security measures are

implemented: - Data Encryption: Use of SSL/TLS protocols for data

transmission.

- User Authentication: Implementation of secure login mechanisms for both patients and healthcare providers.

- Data Access Control: Role-based access to ensure that users can only view data relevant to them.

3. Implementation of Artificial Intelligence

3.1 User Interface

Integration Point: On the web dashboard for both patients and doctors.

AI Functionality:

- Personalized Insights: Provide users with tailored health recommendations based on their individual data and risk factors.

- Health Alerts: Use AI to generate alerts for abnormal readings or potential health risks, helping patients and doctors take timely action.

3.2 Continuous Learning

Integration Point: Throughout the system's operation.

- AI Functionality: Implement a feedback loop where the AI models learn from new data over time. This can help improve the accuracy and reliability of predictions and insights.

- Reinforcement Learning: The system can adapt its recommendations based on user responses and outcomes, continually improving its efficacy.

4. Conclusion

Important fact that came under notice while selecting project design is that all the circuit components used in the remote health detection system are available easily. With the tremendous development in the integrated circuit industry, Microelectromechanical Systems (MEMs) and microcontrollers have become affordable; have increased processing speeds, miniaturized and power efficient. This led to increased development of embedded systems that the healthcare specialists are adopting. Same embedded

systems have also been adapting in the Smartphone technology. In addition, with increased internet usage in most developing countries through mobile phones, and with use of Internet of things (IoT) will become adaptable at a faster rate. This Remote e-Health Care system uses these concepts of new technology to come up with a better system for the improved quality of life for people in society .

References

[1] : GUPTA, Dev; PARIKH, Akanksha; SWARNALATHA, R.. Integrated healthcare monitoring device for obese adults using internet of things (IoT). International Journal of Electrical and Computer Engineering (IJECE), [S.l.], v. 10, n. 2, p. 1239-1247, apr. 2020. ISSN 2722-2578. Available at: <https://ijece.iaescore.com/index.php/IJECE/article/view/20240>

[2] : ALAMSYAH, Alamsyah et al. Internet of things–based vital sign monitoring system. International Journal of Electrical and Computer Engineering (IJECE), [S.l.], v. 10, n. 6, p. 5891-5898, dec. 2020. ISSN 2722-2578. Available at: <https://ijece.iaescore.com/index.php/IJECE/article/view/21431>

[3] : Kishor A., Chakraborty C. Artificial Intelligence and Internet of Things Based Healthcare 4.0 Monitoring System. [(accessed on 5 October 2022)];*Wirel. Pers. Commun.* 2021 :1–17. doi: 10.1007/s11277-021-08708-5.

[4] : Acharya A.D., Patil S.N. IoT based Health Care Monitoring Kit; Proceedings of the 2020 Fourth International Conference on Computing Methodologies and Communication (ICCMC); Erode, India. 11–13 March 2020; pp. 363–368.

CODING PART : (where the sensors are used to take the respective data for the patient and then that data is transmitted to cloud server and the following alert according to patient health condition is transmitted back to user)

```
import time
import logging
import requests
import json
from cryptography.fernet import Fernet
import Adafruit_DHT  # For DHT22 Temperature sensor
from max30100 import MAX30100  # MAX30100 pulse oximeter library
import random  # For simulating blood pressure
from gpiozero import Button  # Example for integrating a button (for user control)
# Constants for health thresholds
HEART_RATE_THRESHOLD_HIGH = 100
HEART_RATE_THRESHOLD_LOW = 60
TEMPERATURE_THRESHOLD_HIGH = 38
TEMPERATURE_THRESHOLD_LOW = 35
OXYGEN_LEVEL_THRESHOLD_LOW = 92
BP_SYSTOLIC_THRESHOLD_HIGH = 140
BP_DIASTOLIC_THRESHOLD_HIGH = 90
# Setup logging for real-time health data
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(message)s')
# Encryption key for secure data transmission
key = Fernet.generate_key()
cipher_suite = Fernet(key)
# Cloud API Endpoint (Cloud)
API_KEY = "API_KEY"  # Replace with your actual API key
URL = "url"
# Initialize MAX30100 sensor for heart rate and SpO2 readings
def initialize_max30100():
    """Initialize MAX30100 sensor."""
    max30100 = MAX30100.MAX30100()
    max30100.enable_spo2()
    max30100.set_led_drive(0x03)  # Set LED drive to 50mA
    max30100.set_sample_rate(MAX30100.SAMPLE_RATE_100HZ)  # Set the sample rate
    return max30100
# Read heart rate and oxygen level from MAX30100
def read_heart_rate_and_spo2(max30100):
    """Read heart rate and oxygen level (SpO2) from MAX30100 sensor."""
    try:
        heart_rate, oxygen_level = max30100.read_fifo()
        if heart_rate and oxygen_level:
            logging.info(f"Heart Rate: {heart_rate} bpm")
            logging.info(f"Oxygen Level: {oxygen_level}%")
            return heart_rate, oxygen_level
        else:
            logging.warning("Failed to read valid heart rate or oxygen level.")
            return None, None
```

```
    except Exception as e:
        logging.error(f"Error reading from MAX30100: {e}")
        return None, None
# Simulate reading temperature from a DHT22 sensor
def read_temperature():
    """Simulate reading temperature from a DHT22 sensor."""
    sensor = Adafruit_DHT.DHT22  # DHT22 sensor
    pin = 4  # GPIO pin where your sensor is connected (e.g., GPIO4)
    humidity, temperature = Adafruit_DHT.read_retry(sensor, pin)
    if temperature is None:
        raise ValueError("Failed to read temperature.")
    return temperature
# Simulate reading blood pressure (systolic and diastolic)
def read_blood_pressure():
    """Simulate reading blood pressure from a sensor."""
    systolic = random.randint(80, 240)
    diastolic = random.randint(50, 150)
    return systolic, diastolic
# Get real-time data from sensors
def get_sensor_data(max30100):
    """Collect real-time data from all sensors."""
    try:
        # Read heart rate and oxygen level from MAX30100
        heart_rate, oxygen_level = read_heart_rate_and_spo2(max30100)
        # Read temperature from DHT22 sensor
        temperature = read_temperature()
        # Read simulated blood pressure values
        systolic, diastolic = read_blood_pressure()
        # Log collected data
        logging.info(f"Temperature: {temperature:.2f} °C")
        logging.info(f"Heart Rate: {heart_rate} bpm")
        logging.info(f"Oxygen Level: {oxygen_level}%")
        logging.info(f"Blood Pressure: {systolic}/{diastolic} mmHg")
        return temperature, heart_rate, oxygen_level, systolic, diastolic
    except Exception as e:
        logging.error(f"Error reading sensor data: {e}")
        return None, None, None, None, None
# Analyze the collected health data based on defined thresholds
def analyze_health_data(heart_rate, temperature, oxygen_level, systolic, diastolic):
    """Analyze health data based on predefined thresholds and generate alerts."""
    alerts = []
    if heart_rate is None or temperature is None or oxygen_level is None or systolic is None or diastolic is
None:
        logging.warning("Incomplete data, skipping analysis.")
        return alerts
    # Heart rate alert
    if heart_rate > HEART_RATE_THRESHOLD_HIGH:
        alerts.append(f"Alert: Heart rate is too high ({heart_rate} bpm).")
    elif heart_rate < HEART_RATE_THRESHOLD_LOW:
        alerts.append(f"Alert: Heart rate is too low ({heart_rate} bpm).")
    # Temperature alert
    if temperature > TEMPERATURE_THRESHOLD_HIGH:
        alerts.append(f"Alert: Body temperature is high ({temperature:.2f} °C), possible fever.")
```

```
    if temperature < TEMPERATURE_THRESHOLD_LOW:
        alerts.append(f"Alert: Body temperature is low ({temperature:.2f} °C), possible hypothermia.")
    # Oxygen level alert
    if oxygen_level < OXYGEN_LEVEL_THRESHOLD_LOW:
        alerts.append(f"Alert: Oxygen level is low ({oxygen_level}%).")
    # Blood pressure alert
    if systolic > BP_SYSTOLIC_THRESHOLD_HIGH or diastolic > BP_DIASTOLIC_THRESHOLD_HIGH:
        alerts.append(f"Alert: High blood pressure detected! Systolic: {systolic} mmHg, Diastolic: {diastolic}
mmHg.")
    if not alerts:
        alerts.append("All health parameters are within normal range.")
    return alerts
# Display health status and alerts
def display_health_status(heart_rate, temperature, oxygen_level, systolic, diastolic, alerts):
    """Display the health data and generated alerts."""
    logging.info("\nHealth Data:")
    logging.info(f"Heart Rate: {heart_rate} bpm")
    logging.info(f"Temperature: {temperature:.2f} °C")
    logging.info(f"Oxygen Level: {oxygen_level}%")
    logging.info(f"Blood Pressure: {systolic}/{diastolic} mmHg")
    logging.info("\nHealth Status:")
    for alert in alerts:
        logging.warning(alert)
# Securely encrypt data before sending to the cloud
def encrypt_data(data):
    """Encrypt the health data before sending it to the cloud."""
    encrypted_data = cipher_suite.encrypt(data.encode())
    return encrypted_data
# Send encrypted data to cloud
def send_data_to_cloud(temperature, heart_rate, oxygen_level, systolic, diastolic):
    """Send the collected data to cloud."""
    payload = {
        "api_key": API_KEY,
        "field1": temperature,
        "field2": heart_rate,
        "field3": oxygen_level,
        "field4": systolic,
        "field5": diastolic,
    }
    # Encrypt the payload before transmission
    encrypted_payload = encrypt_data(json.dumps(payload))
    # Send the data to cloid
    response = requests.post(URL, data={"encrypted": encrypted_payload})
    if response.status_code == 200:
        logging.info("Data sent to Cloud successfully!")
    else:
        logging.error("Failed to send data to Cloud")
# Main loop for collecting data, analyzing, and sending to the cloud
def main():
    """Main loop for the health monitoring system."""
    logging.info("Advanced IoT Healthcare Monitoring System Started\n")
    # Initialize the MAX30100 sensor
    max30100 = initialize_max30100()
```

```
    while True:
        try:
            # Collect real-time sensor data
            temperature, heart_rate, oxygen_level, systolic, diastolic = get_sensor_data(max30100)
            if None in [temperature, heart_rate, oxygen_level, systolic, diastolic]:
                logging.warning("Invalid data entered. Skipping this cycle.")
                continue
            # Analyze health data based on thresholds
            alerts = analyze_health_data(heart_rate, temperature, oxygen_level, systolic, diastolic)
            # Display health status and alerts
            display_health_status(heart_rate, temperature, oxygen_level, systolic, diastolic, alerts)
            # Send data to ThingSpeak
            send_data_to_thingspeak(temperature, heart_rate, oxygen_level, systolic, diastolic)
            # Ask the user if they want to continue monitoring
            continue_monitoring = input("\nDo you want to continue monitoring? (yes/no): ").strip().lower()
            if continue_monitoring != 'yes':
                logging.info("Monitoring stopped by user.")
                break
            # Wait before next cycle
            time.sleep(5)  # Adjust sleep time as needed to control data collection frequency
        except KeyboardInterrupt:
            logging.info("Monitoring stopped by user.")
            break
if __name__ == "__main__":
    main()
```